

作业HW6*实验报告

姓名: 李盛鹏

学号: 2351136

日期: 2025年3月2日

实验报告格式要求

- 按照模板（使用Markdown等也请保证报告内包含模板中的要素）
 - 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考vscode配色）
 - 实验报告要求在文字简洁的同时将内容表示清楚
 - 报告内不要大段贴代码，尽量控制在20页以内
-

1. 涉及数据结构和相关背景

1. 向量（vector） vector是 C++ 标准库中的一个动态数组容器，它可以根据需要自动调整大小。在这些代码中，vector被广泛用于存储数组元素、结果集等。例如在三数之和.cpp中，使用vector<vector>来存储所有满足条件的三元组，使用vector来存储输入的数组。
 2. 优先队列（priority_queue） priority_queue是 C++ 标准库中的一个容器适配器，它提供了一种基于堆的数据结构，默认情况下是最大堆。在序列.cpp中，使用priority_queue来维护前 n 小的数，通过比较元素的大小，保证堆顶元素是最大的，当堆的大小超过 n 时，弹出堆顶元素，从而始终保持堆中元素是前 n 小的数。
 3. 字符串（string） string是 C++ 标准库中的一个字符串类，它提供了方便的字符串操作。在最大数.cpp中，使用string来存储最终拼接后的最大数字。
-

2. 实验内容

2.1 求逆序对数

2.1.1 问题描述

问题描述 给定一个长度为 N 的整数序列 A，求其中逆序对的数量。逆序对的定义是：对于序列中的两个元素 $A[i]$ 和 $A[j]$ ，如果满足 $i < j$ 且 $A[i] > A[j]$ ，则称 (i, j) 为一个逆序对。

输入：

多组测试数据，每组数据包含两行：第一行为整数 NN ($1 \leq N \leq 20000$)，当输入 0 时结束。第二行为 N 个整数，表示序列 A。输出：

每组数据对应一行，输出逆序对的个数。

2.1.2 基本要求

1. 时间复杂度：由于 N 的最大值为 20000，算法的时间复杂度应控制在 $O(N \log N)$ 以内。
2. 空间复杂度：需要额外的空间来存储临时数组，空间复杂度为 $O(N)$ 。

3. 正确性：算法需要正确计算逆序对的数量。

2.1.3 数据结构设计

std::vector 用途：在main函数中，vector arr(n)用于存储用户输入的整数序列。这个向量的大小根据用户输入的n来确定，用于后续计算逆序对数的操作。vector temp(arr)是一个临时向量，作为辅助空间用于归并排序过程中暂存数据。在Merge函数中，会将arr的部分数据复制到temp中，然后再根据比较结果将temp中的数据按顺序放回arr中，从而完成归并操作。

2.1.4 功能说明（函数、类）

```
function MergeSort(arr, low, high, temp):
    if low == high:
        return 0
    mid = (low + high) / 2
    result = 0
    // 递归排序左半部分
    result = result + MergeSort(arr, low, mid, temp)
    // 递归排序右半部分
    result = result + MergeSort(arr, mid + 1, high, temp)
    // 合并两个已排序的子数组
    result = result + Merge(arr, low, mid, high, temp)
    return result
```

```
function Merge(arr, low, mid, high, temp):
    i = low
    j = mid + 1
    k = low
    result = 0
    // 复制当前数组到临时数组
    for idx from low to high:
        temp[idx] = arr[idx]
    // 合并两个子数组
    while i <= mid and j <= high:
        if temp[i] <= temp[j]:
            arr[k] = temp[i]
            i = i + 1
            k = k + 1
        else:
            // 发现逆序对
            result = result + (mid - i + 1)
            arr[k] = temp[j]
            j = j + 1
            k = k + 1
    // 处理剩余元素
    while i <= mid:
        arr[k] = temp[i]
        i = i + 1
        k = k + 1
```

```
while j <= high:
    arr[k] = temp[j]
    j = j + 1
    k = k + 1
return result
```

2.1.5 调试分析（遇到的问题和解决方法）

问题 1：如何在归并排序过程中统计逆序对数？在归并排序中，当合并两个已排序的子数组时，如果发现左边子数组的当前元素大于右边子数组的当前元素，那么左边子数组中从当前元素到中间元素的所有元素都与右边子数组的当前元素构成逆序对。解决方法：在 Merge 函数中，当 $\text{temp}[i] > \text{temp}[j]$ 时，说明 $\text{temp}[i]$ 及其后面的元素（直到 mid ）都与 $\text{temp}[j]$ 构成逆序对，所以逆序对的数量增加 $\text{mid} - i + 1$ 。

问题 2：如何处理不同长度的输入序列？程序需要能够处理不同长度的输入序列，并且每次处理完一个序列后能够继续处理下一个序列，直到遇到无效输入（序列长度小于等于 0）。解决方法：使用 `while (cin >> n)` 循环不断读取序列的长度 n ，当 $n \leq 0$ 时退出程序。对于每个有效的 n ，读取 n 个整数到数组中进行逆序对数的计算。#### 2.1.6 总结和体会

2.2 最大数

2.2.1 问题描述

给定一个整数数组 `nums`，需要将数组中的数字重新排列，使得拼接后的数字字符串表示的数值最大，最终返回这个最大的数字字符串。例如，对于数组 `[3, 30, 34, 5, 9]`，经过重新排列后应得到表示最大数值的字符串。

2.2.2 基本要求

输入：一个整数数组 `nums`，数组中的元素为非负整数。输出：一个字符串，表示将数组中的数字重新排列后能得到的最大数字。实现思路：需要对数组中的元素进行排序，排序规则不是简单的数值大小比较，而是要比较两个数字拼接后的字符串大小。比如比较 3 和 30，需要比较 "330" 和 "303" 的大小，因为 "330" > "303"，所以在排序时 3 应该排在 30 前面。

2.2.3 数据结构设计

```
// 数据结构设计
// 主要的数据结构是一个整数向量 nums，用于存储输入的整数集合
// 还会用到一个字符串 result，用于存储最终拼接成的最大数字字符串

// 输入数据结构
nums: 整数向量 // 存储输入的整数集合

// 输出数据结构
result: 字符串 // 存储最终拼接成的最大数字字符串
```

2.2.4 功能说明（函数、类）

```
// 类定义
class Solution {
    // 主函数，用于计算最大数字字符串
    function largestNumber(nums: 整数向量): 字符串 {
        // 对 nums 进行自定义排序
        sort(nums, 自定义比较函数)

        // 处理特殊情况：如果排序后的第一个元素是 0，则直接返回 "0"
        if (nums[0] == 0) {
            return "0"
        }

        // 初始化结果字符串
        result = ""

        // 遍历排序后的 nums，将每个数字转换为字符串并拼接到 result 中
        for each num in nums {
            result += 字符串(num)
        }

        return result
    }

    // 自定义比较函数，用于确定两个数字 x 和 y 的排序顺序
    function 自定义比较函数(x: 整数, y: 整数): 布尔值 {
        // 将 x 和 y 分别转换为字符串并拼接
        str_x_y = 字符串(x) + 字符串(y)
        str_y_x = 字符串(y) + 字符串(x)

        // 比较拼接后的字符串大小
        return str_x_y > str_y_x
    }
}
```

2.2.5 调试分析（遇到的问题和解决方法）

遇到的问题

- 排序规则的确定：普通的排序函数（如 sort）默认是按照数值大小进行排序的，这不符合本题要求。本题需要根据数字拼接后的字符串大小来确定排序顺序，所以需要自定义排序规则。前导零的处理：如果数组中的所有数字都是 0，那么最终结果应该是 "0"，而不是多个 "0" 拼接的字符串，如 "000"。

解决方法

- 自定义排序规则：使用 C++ 的 sort 函数，并传入一个自定义的比较函数。这里使用 lambda 表达式来实现自定义比较函数。对于两个数字 x 和 y，比较 to_string(x) + to_string(y) 和 to_string(y) + to_string(x) 的大小，如果前者大于后者，则 x 排在 y 前面。

2.2.6 总结和体会

- 排序规则的重要性：在解决此类问题时，排序规则的选择至关重要。不能仅仅依赖于默认的排序规则，需要根据具体问题的要求进行自定义。
- 边界情况的处理：要特别注意边界情况，如本题中的前导零问题。如果不处理前导零，可能会导致结果不符合预期。
- lambda 表达式的便利性：在 C++ 中，lambda 表达式为我们提供了一种简洁的方式来定义匿名函数，非常适合用于自定义排序规则等场景，使得代码更加简洁易懂。

2.3 排序

2.3.1 问题描述

实现一个排序算法，具体是对给定的整数向量进行排序。要求使用快速排序算法完成排序任务，最终返回排序好的整数向量。

2.3.2 基本要求

- 输入：一个整数向量 `list`，该向量包含需要排序的整数元素。
- 输出：返回排序好的整数向量。
- 排序算法：使用快速排序算法对输入的向量进行排序。快速排序是一种分治算法，其基本思想是选择一个基准元素，将数组分为两部分，使得左边部分的元素都小于等于基准元素，右边部分的元素都大于基准元素，然后分别对左右两部分递归地进行排序。

2.3.3 数据结构设计

- `std::vector`：这是一个标准库提供的动态数组容器，用于存储待排序的整数序列。`std::vector` 提供了随机访问的能力，并且可以根据需要动态调整大小。其底层实现通常是一个连续的内存块，这使得通过索引访问元素的时间复杂度为 $O(1)$ 。

```
vector<int> list; // 用于存储待排序的整数序列
```

2.3.4 功能说明（函数、类）

```
vector<int> mySort(vector<int>& list)
{
    int n = list.size();
    quickSort(list, 0, n - 1);
    return list;
}

void quickSort(vector<int>& arr, int low, int high)
{
    if (low < high)
    {
        int pivot = partition(arr, low, high);
        quickSort(arr, low, pivot - 1);
    }
}
```

```
        quickSort(arr, pivot + 1, high);
    }
}

int partition(vector<int>& arr, int low, int high)
{
    int randomIdx = low + rand() % (high - low + 1);
    swap(arr[low], arr[randomIdx]);
    int pivot = arr[low];

    int left = low + 1, right = high;
    while (true)
    {
        while (left <= right && arr[left] <= pivot) left++;
        while (left <= right && arr[right] > pivot) right--;

        if (left > right) break;
        swap(arr[left], arr[right]);
        left++;
        right--;
    }

    swap(arr[low], arr[right]);
    return right;
}
```

2.3.5 调试分析（遇到的问题和解决方法）

问题 1：基准元素的选择

- 问题描述：在快速排序中，基准元素的选择会影响排序的性能。如果每次都选择固定位置的元素作为基准元素，对于某些特殊的输入序列（如已经有序的序列），快速排序的性能会退化为 $O(n^2)$ 。
- 解决方法：采用随机选择基准元素的方法。在 partition 函数中，通过 `int randomIdx = low + rand() % (high - low + 1);` 随机选择一个索引作为基准元素的位置，然后将该位置的元素与 low 位置的元素交换，这样可以在一定程度上避免最坏情况的发生，提高算法的平均性能。

问题 2：分区操作的实现

- 问题描述：如何正确地将数组分为两部分，使得左边部分的元素都小于等于基准元素，右边部分的元素都大于基准元素。
- 解决方法：在 partition 函数中，使用两个指针 left 和 right 分别从数组的两端向中间移动。left 指针从 low + 1 开始，找到第一个大于基准元素的元素；right 指针从 high 开始，找到第一个小于等于基准元素的元素。如果 left 小于等于 right，则交换这两个元素，然后继续移动指针，直到 left 大于 right。最后将基准元素放到正确的位置（即 right 位置）。

2.3.6 总结和体会

- 快速排序的优点：快速排序是一种高效的排序算法，平均时间复杂度为 $O(n \log n)$ ，在大多数情况下表现良好。它是一种原地排序算法，不需要额外的存储空间（除了递归调用所需的栈空间）。

- 随机化的重要性：通过随机选择基准元素，可以有效地避免快速排序在最坏情况下的性能退化。在实际应用中，对于可能出现特殊输入序列的情况，随机化是一种简单而有效的优化方法。
- 递归的使用：快速排序是一种递归算法，通过递归地对左右两部分进行排序，实现了对整个数组的排序。在实现递归算法时，需要注意递归的终止条件，避免出现无限递归的情况。在本代码中，递归的终止条件是 $low < high$ ，当 low 大于等于 $high$ 时，递归停止。

2.4 排序

2.4.1 问题描述

给定 m 个数字序列，每个序列包含 n 个非负整数。从每个序列中选取一个数字组成新序列，总共可构造出 (n^m) 个新序列。对每个新序列中的数字求和，会得到 (n^m) 个和，需要找出其中最小的 n 个和。输入包含多个测试用例，第一行是测试用例的数量 T ，每个测试用例第一行是 m 和 n ，接下来 m 行每行有 n 个数表示序列。输出是每组测试用例的最小 n 个和，用空格隔开。

2.4.2 基本要求

- 输入的第一行读取一个整数 T ，表示测试用例的数量。
- 对于每个测试用例，读取两个正整数 m 和 n ，其中 $(0 < m \leq 100)$ ， $(0 < n \leq 2000)$ 。
- 接着读取 m 行，每行有 n 个不大于 10000 的非负整数，表示 m 个序列。
- 计算所有可能的 (n^m) 个新序列的和，找出其中最小的 n 个和。
- 对于每组测试用例，输出一行，包含最小的 n 个和，数之间用空格隔开。

2.4.3 数据结构设计

1. 数组 `ans` 数组：类型：`int ans[MAXN]`，其中 `MAXN` 被定义为 2010。用途：用于存储前 n 个最小和。在程序运行过程中，会不断更新这个数组，最终存储的是经过多次合并操作后得到的前 n 个最小和。
`new_arr` 数组：类型：`int new_arr[MAXN]`。用途：用于存储每一轮新输入的数组，这个数组会和 `ans` 数组进行合并操作，以得到新的前 n 个最小和。
2. 优先队列（堆）`priority_queue q`：类型：`priority_queue` 是 C++ 标准库中的容器适配器，默认是大顶堆（堆顶元素是最大的）。用途：在 `maintain_sum` 函数中，用于维护前 n 个最小和。在计算新的和时，利用优先队列的特性，将和加入队列并保持队列大小不超过 n ，从而方便找出前 n 个最小和。

2.4.4 功能说明（函数、类）

`maintain_sum` 函数

```
/**
 * @brief 生成一个新的数组，找出当前  $n$  个最小和
 * @param ans 已经排序好的前  $n$  个最小和
 * @param new_arr 新输入的数组
 * @param n 每个数组的元素个数
 */
void maintain_sum(int* ans, int* new_arr, int n)
```

2.4.5 调试分析（遇到的问题和解决方法）

问题 1：如何存储并找出最小的 n 个和

- 所有可能的新序列和的数量是 (n^m) ，当 m 和 n 较大时，这个数量会非常大，无法直接存储所有的和再排序。
- 解决方法：**使用优先队列（最大堆）来维护最小的 n 个和。在计算新的和时，如果堆的大小小于 n ，直接将新和加入堆中；如果堆的大小等于 n 且新和小于堆顶元素（当前堆中最大的元素），则弹出堆顶元素，将新和加入堆中。

问题 2：计算复杂度高

- 直接计算所有 (n^m) 个新序列的和的时间复杂度非常高。
- 解决方法：**在每次处理新序列时，只计算当前序列与之前已得到的最小 n 个和的组合，而不是计算所有可能的组合。在 `maintain_sum` 函数中，通过两层循环遍历之前的最小 n 个和与当前序列的元素，计算新的和，并使用优先队列维护最小的 n 个和。

2.4.6 总结和体会

- 数据结构的重要性：**优先队列在处理这类需要维护最小或最大的 k 个元素的问题中非常有用。它可以在 $(O(\log k))$ 的时间复杂度内完成插入和删除操作，避免了对所有元素进行排序的高复杂度。
- 算法优化的必要性：**通过对输入序列进行排序，利用有序性可以减少不必要的计算，从而降低时间复杂度。在处理大规模数据时，算法的优化可以显著提高程序的运行效率。
- 分步骤解决问题：**将问题分解为多个小步骤，如每次处理一个新序列，逐步计算最小的 n 个和，避免一次性计算所有可能的组合，降低了问题的复杂度。

2.5 排序

2.5.1 问题描述

给定一个整数数组 `nums`，需要找出其中所有满足以下条件的三元组 `[nums[i], nums[j], nums[k]]`：

- 索引 i 、 j 、 k 互不相等，即 $i \neq j$ 、 $i \neq k$ 且 $j \neq k$ 。
- 三元组元素之和为 0，即 $nums[i] + nums[j] + nums[k] == 0$ 。输入格式为 2 行，第 1 行是一个整数，表示数组元素个数；第 2 行是一组整数，中间以空格分隔。输出要求为所有和为 0 的三元组，每个三元组占一行，元素之间以空格分隔，且每个三元组内元素按从小到大顺序排列，所有三元组按三元组中最小元素从小到大的顺序依次打印，同时答案中不能包含重复的三元组。

2.5.2 基本要求

- 输入处理：**正确读取输入的数组元素个数和数组元素。
- 查找三元组：**找出所有满足和为 0 的三元组。
- 去重：**确保结果中不包含重复的三元组。
- 排序输出：**每个三元组内元素按从小到大排序，所有三元组按三元组中最小元素从小到大排序。

2.5.3 数据结构设计


```
vector<int> nums; // 存储输入的整数数组
vector<vector<int>> result; // 存储满足条件的三元组
```

2.5.4功能说明（函数、类）

```
// 定义一个名为 Solution 的类
类 Solution:
    // 定义一个公共方法 threeSum, 用于找出数组中所有和为 0 的不重复三元组
    方法 threeSum(数组 nums):
        // 获取数组的长度
        n = nums 的长度
        // 如果数组长度小于等于 2, 直接返回空的二维数组
        如果 n <= 2:
            返回空的二维数组

        // 对数组进行排序
        对 nums 进行排序

        // 定义一个二维数组 result 用于存储结果
        初始化二维数组 result

        // 遍历数组, 固定第一个数
        对于 i 从 0 到 n - 2:
            // 跳过重复的 nums[i]
            如果 i > 0 且 nums[i] 等于 nums[i - 1]:
                继续下一次循环

            // 初始化左右指针
            left = i + 1
            right = n - 1

            // 双指针移动
            当 left < right 时:
                // 计算三数之和
                sum = nums[i] + nums[left] + nums[right]
                // 如果和小于 0, 左指针右移
                如果 sum < 0:
                    left 加 1
                // 如果和大于 0, 右指针左移
                否则如果 sum > 0:
                    right 减 1
                // 如果和等于 0, 找到一个符合条件的三元组
                否则:
                    // 将三元组添加到结果数组中
                    将 {nums[i], nums[left], nums[right]} 添加到 result 中

                    // 跳过重复的 nums[left]
                    当 left < right 且 nums[left] 等于 nums[left + 1]:
                        left 加 1
                    // 跳过重复的 nums[right]
```

```
        当 left < right 且 nums[right] 等于 nums[right - 1]:
            right 减 1

        // 移动指针
        left 加 1
        right 减 1

    // 返回结果数组
    返回 result
```

2.5.5 调试分析 (遇到的问题和解决方法)

1. 如何找出所有和为 0 的三元组:

- **问题:** 直接使用三重循环遍历数组元素来查找三元组的时间复杂度为 $(O(n^3))$, 效率较低。
- **解决方法:** 采用双指针法。首先对数组进行排序, 时间复杂度为 $(O(n \log n))$ 。然后固定一个元素 $nums[i]$, 使用两个指针 $left$ 和 $right$ 分别指向 i 之后的元素和数组末尾元素, 通过比较 $nums[i] + nums[left] + nums[right]$ 与 0 的大小来移动指针, 时间复杂度为 $(O(n^2))$ 。总体时间复杂度降为 $(O(n^2))$ 。

•

2. 如何避免结果中出现重复的三元组:

- **问题:** 在查找过程中可能会得到重复的三元组。
- **解决方法:**
 - 对于固定的元素 $nums[i]$, 如果 $i > 0$ 且 $nums[i] == nums[i - 1]$, 则跳过当前元素, 避免重复计算。
 - 当找到一个和为 0 的三元组后, 移动 $left$ 和 $right$ 指针时, 跳过与当前元素相同的元素。

2.5.6 总结和体会

- 算法的重要性: 在解决这类查找问题时, 选择合适的算法可以大大提高程序的效率。双指针法在有序数组中查找满足特定条件的元素组合时非常有效, 通过减少不必要的遍历, 将时间复杂度从 $(O(n^3))$ 降低到 $(O(n^2))$ 。
- 去重的处理: 在处理数组问题时, 去重是一个常见的需求。通过在合适的位置进行判断和跳过重复元素, 可以避免结果中出现重复的组合。
- 排序的作用: 排序不仅可以帮助我们按要求输出结果, 还为双指针法的使用提供了前提条件, 使得指针的移动具有明确的方向。

3. 实验总结

本次实验涉及多个 C++ 程序, 包括求解最大数、三数之和、序列相关计算、快速排序以及求逆序对数等问题。在求解最大数的程序中, 通过自定义的 `lambda` 表达式作为 `sort` 函数的比较函数, 巧妙地对数字组合后的字符串大小进行比较, 从而实现将数字按特定规则排序以得到最大数, 需注意处理首位为 0 的特殊情况。三数之和的程序运用双指针法, 先对数组排序, 然后通过固定一个数, 利用双指针在剩余数组中寻找满足和为 0 的三元组, 同时利用条件判断跳过重复元素, 避免结果重复。序列相关计算程序中, `maintain_sum` 函数使用优先队列来维护前 n 小的和, 时间复杂度较高, 需要对输入数组进行排序, 通过不断更新优先队列中的元素得到结果。快速排序的程序实现了快速排序算法, 通过随机选择基准元素, 利用 `partition` 函数将数组划分为两部分, 递归

地对左右两部分进行排序。求逆序对数的程序基于归并排序的思想，在归并过程中统计逆序对的数量，通过辅助数组temp进行归并操作。总体而言，这些程序展示了不同算法在解决具体问题时的应用，涉及排序、指针操作、递归等多种编程技巧，让我对算法的设计与实现有了更深入的理解，同时也意识到在处理不同问题时，选择合适的数据结构和算法对程序的效率和正确性至关重要，在编写代码时要注重边界条件的处理和代码的健壮性。